

Unit 2

Data Preprocessing and EDA

Data set: attribute types, explanatory variable, outcome variable

Attribute Types

Nominal:

Categories with no inherent order — labels only. Example: blood type, zip code, marital status.

Interval

Numeric, ordered, equal intervals, but no true zero. Example: temperature in °C — "20° is twice 10°" isn't valid.

Ordinal

Categories with a meaningful rank, but unequal or undefined gaps between them. Example: education level, satisfaction rating.

Ratio

Numeric, ordered, equal intervals, with a true zero meaning "none." Example: height or income — 10 kg is truly twice 5 kg.

Explanatory Variable(s)

- Also called independent, predictor, or feature variables — the inputs a model uses to explain or predict an outcome. A data set usually has many. Example: predicting a house's sale price from its square footage, location, and age.

Outcome Variable

Also called the dependent, target, or response variable — the quantity or category being explained or predicted. A supervised task has exactly one. Example: the sale price itself, the value the model is built to estimate.

Data Preprocessing

6 Core tasks stand between a raw, messy data set and one a mining algorithm can use reliably —

- missing data,
- Smooth out noise,
- Handling outliers,
- categorical encoding,
- Feature scaling and
- Normalization, and
- Discretization.

Handling Missing Data

Ignore the tuple: drop records with missing values — simple, but wastes data and can bias results if missingness isn't random.

Fill manually: a domain expert supplies the value — accurate, but slow and impractical at scale.

Fill with a global constant, e.g., "Unknown" or 0 — fast, but risks the model treating the fill as a real category.

Fill with mean or median (overall or by class) — easy, but understates variability and weakens correlations.

Predict the value via regression, decision-tree, or k-NN imputation — most accurate, at higher computational cost.

Smoothing Noisy Data

Binning: partition sorted values into equal-width or equal-frequency bins, then replace each with its bin mean, median, or boundary.

Regression: fit the data to a function (linear or multiple regression) and use the fitted values in place of the noisy ones.

Outlier analysis: cluster similar values together; values outside any cluster are flagged as noise.

Combined inspection: let automated checks flag suspicious values, then defer the final call to an expert.

Example: sensor readings that jitter ± 0.5 °C around a true trend — binning or regression recovers the signal.

Handling Outliers

Statistical methods: flag points beyond a threshold, e.g., more than $1.5 \times \text{IQR}$ from the quartiles, or $|z\text{-score}| > 3$ — simple and interpretable, but assumes a roughly known distribution.

Clustering-based detection: points that fall outside any cluster are treated as outliers — flexible for multi-dimensional data.

Visualization: box plots and scatter plots surface extreme points for human review.

Treatment: remove, cap (winsorize), transform (e.g., take the log), or keep and model separately — the choice depends on whether the outlier is an error or a real event worth preserving.

Encoding Categorical Variables

One-hot encoding: a binary column per category — no false order implied, but adds many columns at high cardinality.

Label (ordinal) encoding: maps categories to integers — compact, but implies an order that may not exist for nominal data.

Binary / hashing encoding: categories as binary codes or hashed buckets — more compact than one-hot at high cardinality.

Target / frequency encoding: replaces a category with an outcome statistic — powerful, but risks leaking outcome information.

Feature Scaling

Min-Max Scaling

Rescales values into a fixed range, typically $[0, 1]$. Preserves the shape of the original distribution and relationships between values, but is sensitive to outliers, which can compress the rest of the data into a narrow band.

Why it matters: distance-based methods (k-NN, k-means) and gradient-based methods (linear/logistic regression, neural networks) let large-range features dominate unless every feature is on a comparable scale.

Standardization (Z-Score)

Centers values to mean 0 and standard deviation 1. Less thrown off by outliers than min-max, but does not bound values to a fixed range — useful when an algorithm assumes roughly normally distributed input.

Normalization

Min-max normalization: $v' = (v - \min) / (\max - \min) \times (\text{new_max} - \text{new_min}) + \text{new_min}$ — maps values into a chosen range; shifts if min or max change with new data.

Z-score normalization: $v' = (v - \text{mean}) / \text{std_dev}$ — centers the attribute around 0 with unit variance.

Decimal scaling: $v' = v / 10^j$, where j is the smallest integer such that every $|v'| < 1$ — normalizes by moving the decimal point.

Purpose: puts attributes measured on different scales — age in years, income in dollars — onto comparable footing before distance- or gradient-based methods are applied.

Discretization

Definition: converts a continuous attribute into a finite set of intervals, each treated as a category or ordinal label.

Binning: equal-width bins (fixed interval size) or equal-frequency bins (equal count per bin).

Histogram analysis: partitions values based on the distribution's peaks and valleys.

Cluster analysis: groups nearby values into intervals via a clustering algorithm.

Entropy-based (supervised): picks split points that best separate class labels.

Example: exact age into bins like "0–18," "19–35," "36–60," "60+" for a classification model.

Preprocessing at a Glance

TASK	ADDRESSES	COMMON TECHNIQUES
Missing data	Absent or unrecorded values	Deletion, mean/median fill, prediction
Noise & outliers	Measurement error and extreme values	Binning, regression, IQR/z-score detection
Categorical encoding	Non-numeric attributes	One-hot, ordinal, target encoding
Scaling, normalization & discretization	Differing scales and continuous ranges	Min-max, z-score, decimal scaling, binning

Introduction to EDA

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize their main characteristics using visual and statistical methods. Introduced by John Tukey in the 1970s, EDA emphasizes the importance of visual exploration and understanding of data before applying machine learning models.

Primary Goals

To understand the underlying structure of the data.

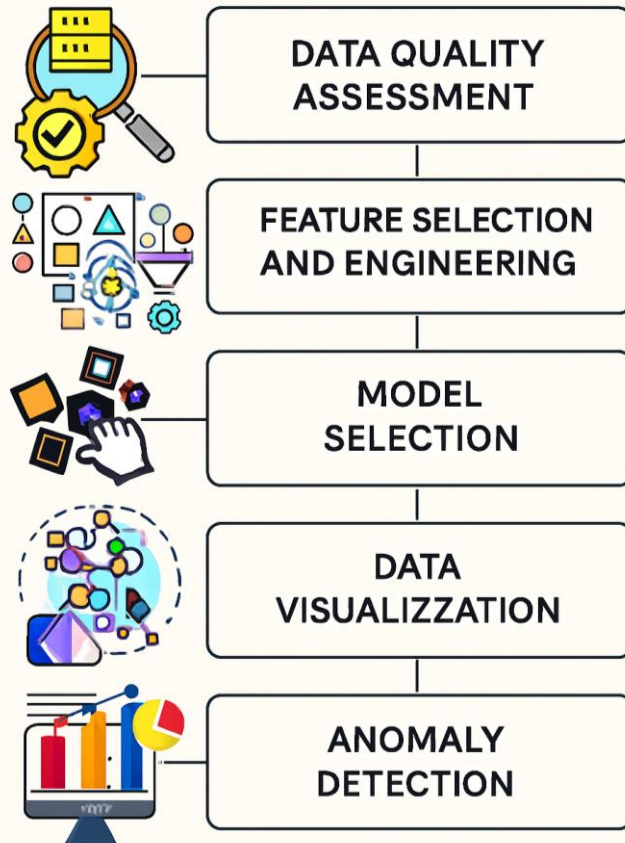
To identify outliers and missing values.

To determine relationships between variables.

To generate hypotheses for further analysis.

EDA Importance in Machine Learning

EDA serves as the foundation for building robust and accurate machine learning models. Here are the key reasons why EDA is indispensable:



EDA helps identify issues such as missing values, duplicate records, and inconsistencies in data.

By analyzing correlations and distributions, EDA aids in selecting relevant features and engineering new ones.

Insights from EDA can influence the choice of machine learning algorithms.

Visualization during EDA communicates complex relationships and patterns effectively.

It highlights unusual data points that could skew model performance.

Key Steps in Exploratory Data Analysis

- Inspecting Data Types
- Checking Data Dimensions
- Viewing Sample

Understanding the Dataset

Summarizing the Data

- Mean, Median, Mode
- Standard Deviation and Variance
- Minimum, Maximum, and Range
- Skewness and Kurtosis

- Handling Missing Values
- Removing Duplicates
- Addressing Outliers

Data Cleaning

Visualizing Data

- Histograms
- Box Plots
- Scatter Plots
- Heatmaps
- Bar Charts and Pie Charts

- Correlation Analysis
- Cross-tabulation
- Pair Plots

Exploring Relationships Between Variables

Visualizing Relationships

- Scaling and Normalization
- Log Transformation
- One-Hot Encoding
- Feature Interaction

Classification Exploratory Data Analysis

- **EDA**

- **Univariate :**

- Graphical EDA: Histograms, Boxplots
 - Non-graphical EDA: Frequency distribution table, measures of central tendency, measures of dispersion

- **Bivariate**

- Graphical EDA
 - Non Graphical EDA

- **Multivariate**

- Graphical EDA: Scatter plot, Heat Map
 - Non-graphical EDA: Cross-tabulation, correlation matrix

Example: EDA in Action

Let's walk through a basic EDA process using Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Load dataset
data = pd.read_csv("data.csv")
```

```
# Summarize data
print(data.info())
print(data.describe())
```

```
# Check for missing values
print(data.isnull().sum())
```

```
# Visualize distributions
sns.histplot(data["column_name"])
plt.show()
```

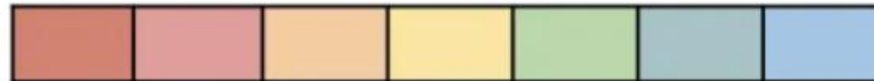
```
# Correlation heatmap
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.show()
```

Feature Selection Techniques

Feature selection is a critical aspect of machine learning that involves choosing the most relevant features from a dataset. It plays a significant role in improving model performance, reducing overfitting, and enhancing interpretability. In this article, I will explore different feature selection techniques, their importance, utility, applicability, and provide code examples to illustrate each technique.

The goal of feature selection is to improve the performance of a machine learning model by reducing the dimensionality of the input data and removing irrelevant or redundant features.

All Features



~~Informative Features~~ Selection



Final Features



Importance of Feature Selection

Importance of Feature Selection in Machine Learning

- 1. Enhanced Model Performance:** Selecting relevant features leads to more accurate and efficient models.
- 2. Reduced Overfitting:** Including irrelevant features can lead to overfitting, where the model performs well on the training data but poorly on test data. Feature selection helps mitigate this issue.
- 3. Improved Interpretability:** Models with fewer features are easier to interpret and explain to stakeholders.
- 4. Faster Training and Inference:** Working with a reduced set of features speeds up both the training and inference phases of the machine learning pipeline.

Types of Feature Selection Techniques:

1. Filter Methods

Filter methods assess the relevance of features based on statistical measures based on their correlation with the target variable. Examples include chi-square test, information gain, and correlation coefficient. Features are selected based on their individual characteristics, without considering the machine learning algorithm used.

For example:

```
from sklearn.feature_selection import SelectKBest, f_classif

# Example of SelectKBest with ANOVA F-value for classification
selector = SelectKBest(score_func=f_classif, k=5)
X_selected = selector.fit_transform(X, y)
```

Types of Feature Selection Techniques:

2. Wrapper Methods

Wrapper methods use a specific machine learning algorithm to evaluate feature importance. They select features based on their impact on model performance. They wrap the feature selection process around the model training and evaluate the model's performance to determine the optimal subset of features. Wrapper method follows a greedy search approach by evaluating all the possible combinations of features against the evaluation criterion. Examples include recursive feature elimination (RFE) i.e. eliminating the feature having lowest coefficient and forward/backward stepwise selection.

For example:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# Recursive Feature Elimination (RFE) with Logistic Regression
estimator = LogisticRegression()
selector = RFE(estimator, n_features_to_select=5)
X_selected = selector.fit_transform(X, y)
```

Types of Feature Selection Techniques:

3. Embedded Methods

Embedded methods incorporate feature selection into the model training process. They select features based on their contribution to the model's predictive power. The feature selection is embedded within the algorithm, and the model selects the most relevant features during training. Examples include LASSO (Least Absolute Shrinkage and Selection Operator) i.e. adding a penalty to the different parameters of the machine learning model to reduce the freedom of the model and tree-based methods like Random Forests, which assign feature importance scores.

For example:

```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier

# SelectFromModel with Random Forest Classifier
selector =
SelectFromModel(RandomForestClassifier(n_estimators=100))
X_selected = selector.fit_transform(X, y)
```

Types of Feature Selection Techniques:

3. Hybrid Methods

Hybrid methods combine aspects of filter, wrapper, and embedded methods. They leverage the strengths of multiple techniques. Dimensionality reduction techniques, such as Principal Component Analysis (PCA) transform the original features into a lower-dimensional space while preserving important information. These methods are effective for reducing the feature space but do not explicitly select individual features.

For example:

```
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression

# Hybrid method combining SelectFromModel and Logistic
Regression
selector = SelectFromModel(LogisticRegression(penalty="l1",
C=0.1))
X_selected = selector.fit_transform(X, y)
```

Applicability of Feature Selection Technique:

Small Datasets: Feature selection is beneficial for datasets with a large number of features relative to the number of samples.

High-Dimensional Data: In cases where the dimensionality of the data is high, feature selection helps in reducing computational complexity and improving model performance.

Interpretability: When interpretability is crucial, selecting a subset of features facilitates easier model understanding and explanation.

- **Definition:** Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize main characteristics before modeling.
- **Purpose:**
 - Identifies patterns and trends
 - Detects outliers and missing values
 - Assists in feature engineering and selection

- Exploratory Data Analysis (EDA) is a fundamental step in ML before modeling. The main objectives include:
- **Understanding data characteristics** (patterns, distributions, missing values).
- **Detecting anomalies, trends, and relationships** to improve model performance.
- **Preparing data** by handling missing values, feature engineering, and scaling.

EDA ensures that we make informed decisions before applying ML algorithms.

Types of EDA

- **Univariate Analysis**
- **Bivariate Analysis**
- **Multivariate Analysis**
- **Missing Value Analysis**
- **Outlier Detection**
- **Feature Engineering Analysis**

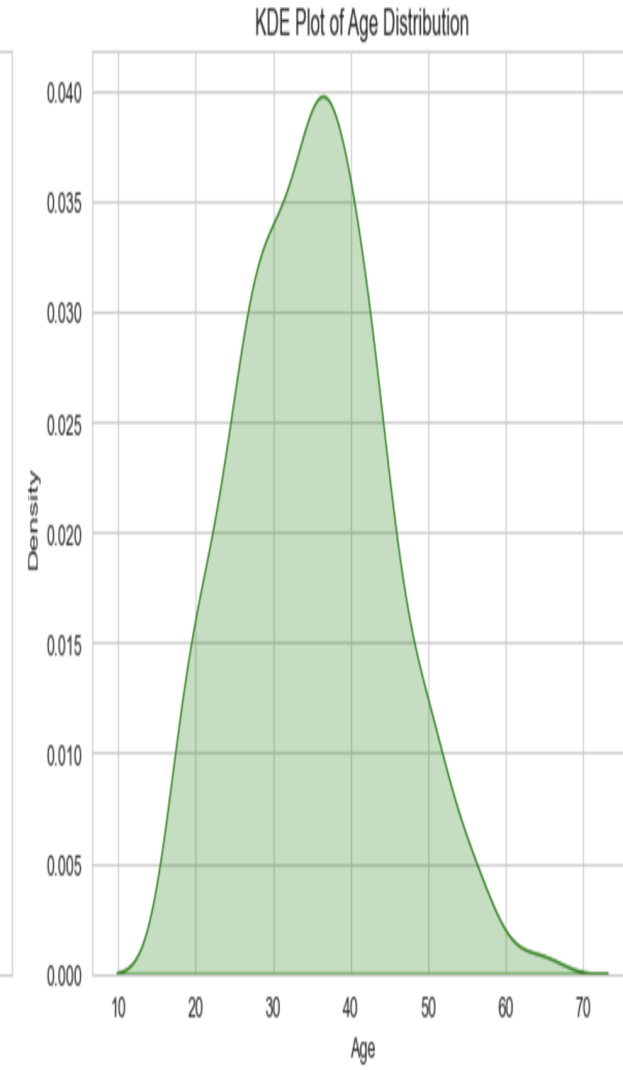
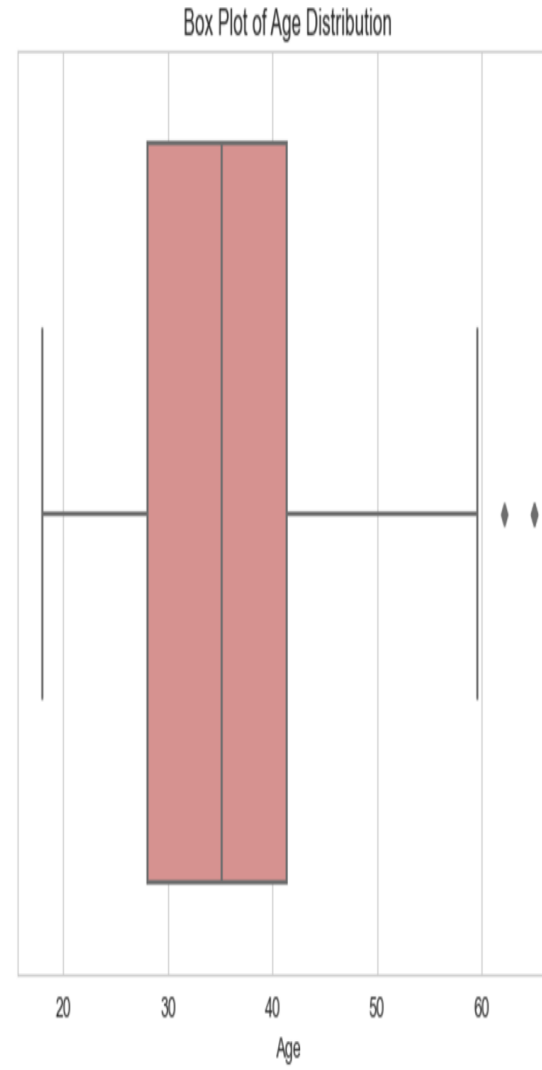
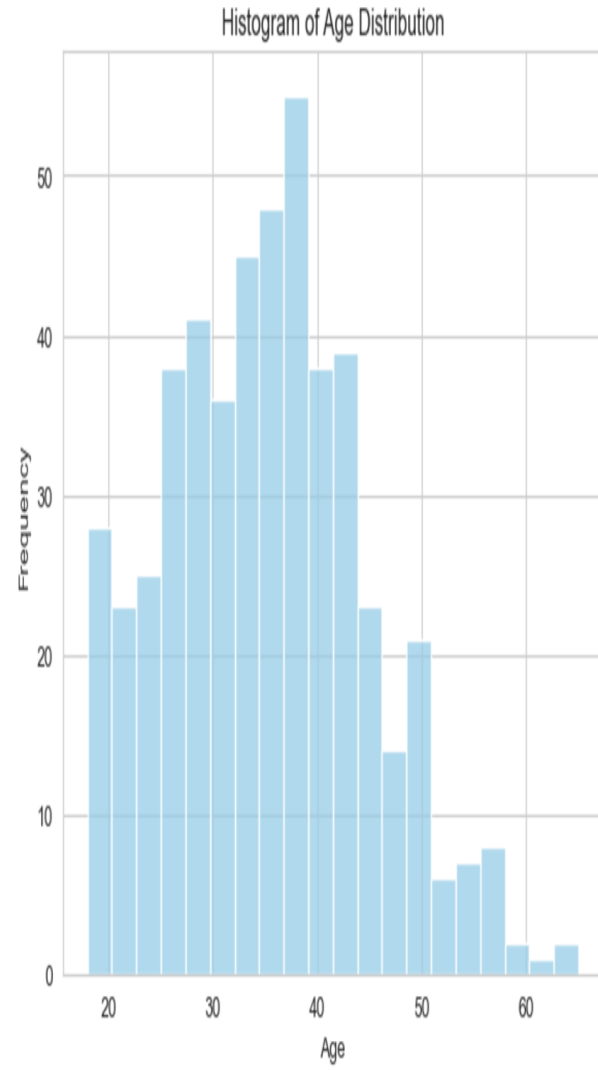
EDA can be divided into different types based on the scope of analysis:

- 1. Univariate Analysis:** Examines single variables.
- 2. Bivariate Analysis:** Analyzes relationships between two variables.
- 3. Multivariate Analysis:** Examines multiple variables together.
- 4. Missing Value Analysis:** Identifies and handles missing data.
- 5. Outlier Detection:** Detects extreme values.
- 6. Feature Engineering Analysis:** Transforms data to improve modeling.
- 7. Time-Series Analysis:** Identifies trends, seasonality, and patterns in temporal data.
- 8. Data Imbalance Analysis:** Checks class distribution in classification problems.

Univariate Analysis

- **Examines single variable distribution**
- **Techniques:**
 - Summary Statistics: Mean, Median, Mode, Skewness
 - Visualizations:
 - Histogram (frequency distribution)
 - Box Plot (outliers)
 - KDE Plot (density estimation)

- Univariate analysis explores the **distribution of a single variable** in a dataset.
- **Common techniques:**
 - **Summary Statistics:** Mean, Median, Mode, Standard Deviation, **Visualizations:**
 - **Histogram:** Shows frequency distribution.
 - **Box Plot:** Detects outliers and spread.
 - **KDE Plot:** Estimates probability density.
- Example: Analyzing age distribution in a dataset using histograms.



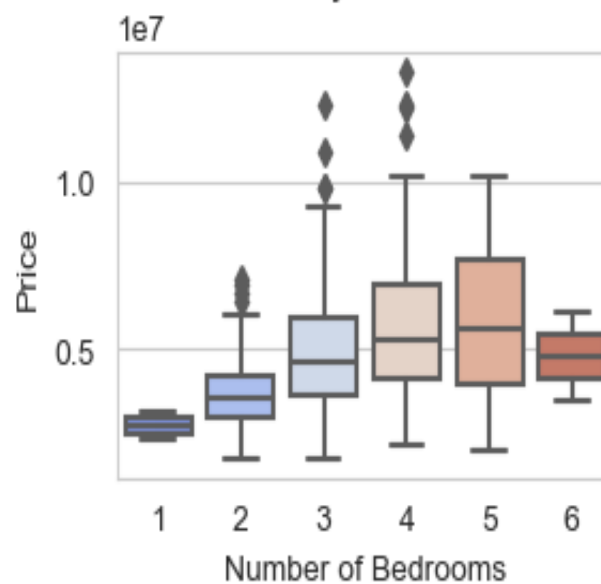
Bivariate Analysis

- **Examines relationships between two variables**
- **Techniques:**
 - **Numerical-Numerical:** Scatter Plot, Correlation Matrix
 - **Categorical-Categorical:** Bar Chart
 - **Numerical-Categorical:** Box Plot

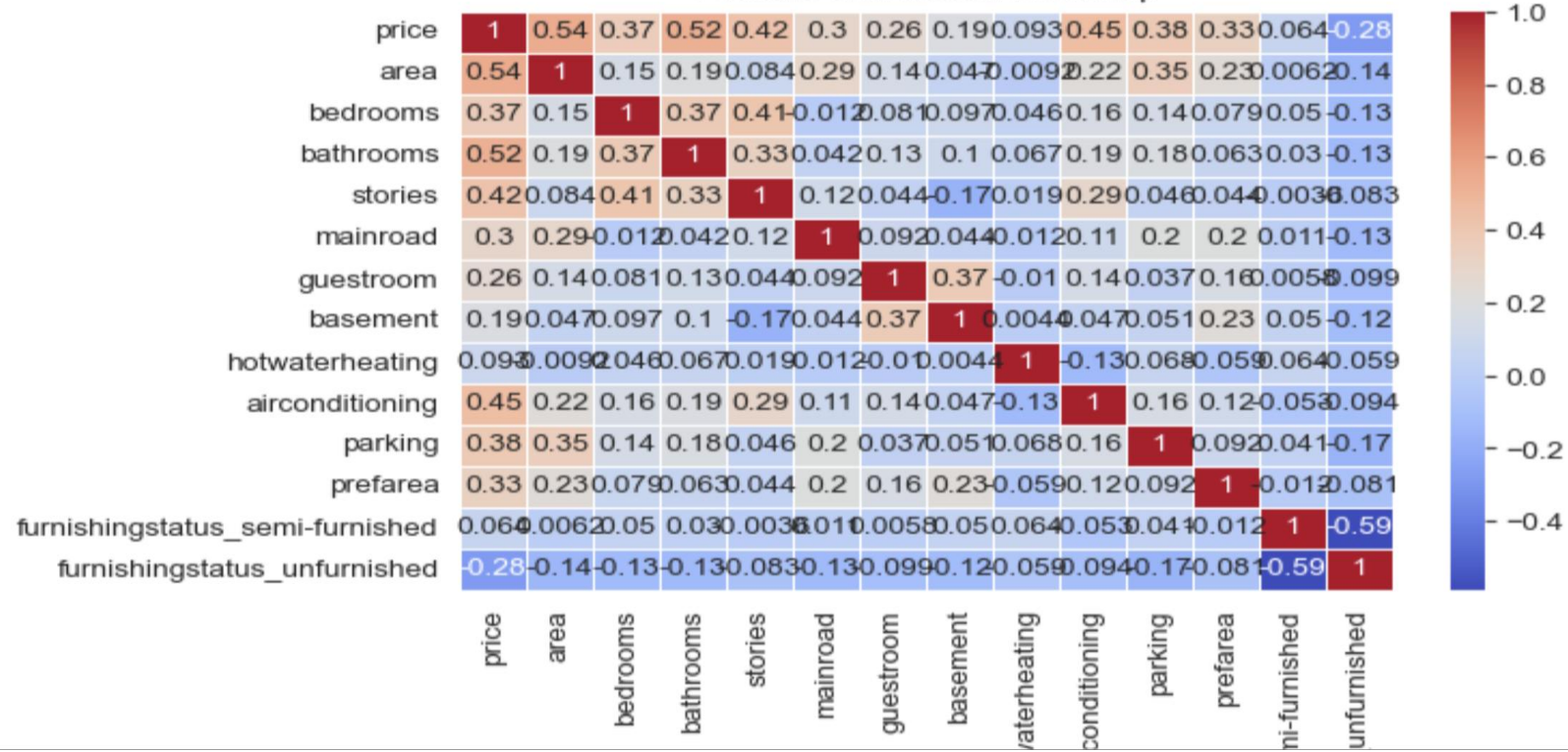
- Bivariate analysis examines **the relationship between two variables** using different approaches:
 - **Numerical-Numerical:**
 - **Scatter Plot:** Visualizes correlations.
 - **Correlation Matrix:** Measures relationships using Pearson/Spearman coefficients.
 - **Categorical-Categorical:**
 - **Bar Charts:** Shows categorical distribution.
 - **Numerical-Categorical:**
 - **Box Plots:** Compares distributions across categories.
- Example: Checking how **house price** varies with **square footage** using a scatter plot.



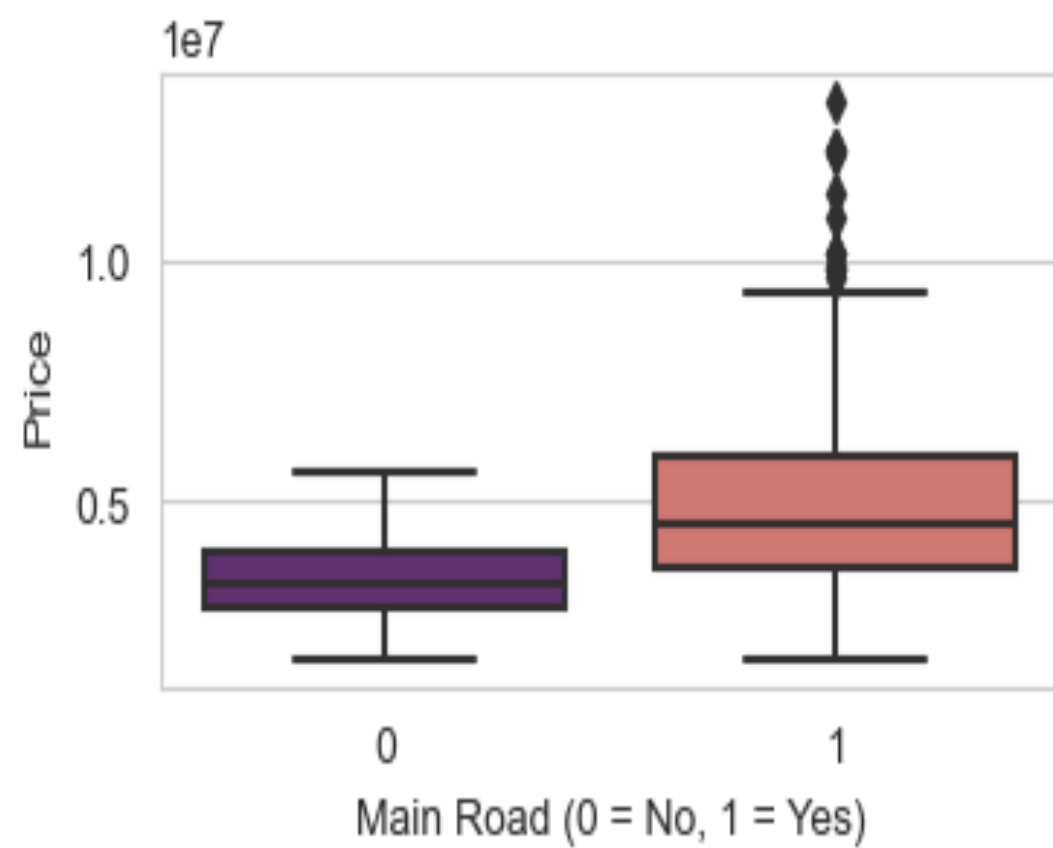
Price Distribution by Number of Bedrooms



Feature Correlation Heatmap



Price Distribution for Houses Near Main Road



Multivariate Analysis

- **Analyzes multiple variables simultaneously**
- **Techniques:**
 - Pair Plot (Seaborn's `sns.pairplot`)
 - Heatmap (correlation matrix visualization)
 - Principal Component Analysis (PCA)

- This analysis evaluates **the interactions between multiple variables** simultaneously.
- **Common techniques:**
 - **Pair Plot:** Visualizes pairwise relationships.
 - **Heatmap:** Displays correlations using a color matrix.
 - **PCA (Principal Component Analysis):** Reduces dimensions for better visualization.
- Example: Finding correlations between **age, salary, and spending habits** in customer datasets.

Missing Value Analysis

- **Identifies and handles missing data**
- **Techniques:**
 - Check missing values (`isnull().sum()`)
 - Visualize missing data (Seaborn Heatmap)
 - Handle missing values:
 - Remove if percentage is low
 - Impute (mean, median, mode, predictive imputation)

- Missing values can distort model predictions, so detecting and handling them is crucial.
- **How to detect missing values?**
 - Use `.isnull().sum()` in Pandas to count missing entries.
 - **Seaborn Heatmap:** Highlights missing data patterns.
- **Handling missing values:**
 - **Drop missing values** if a small percentage is affected.
 - **Impute values:**
 - **Mean/Median/Mode imputation** for numerical data.
 - **Predictive imputation** using regression models.
- Example: Handling missing **income** values in a customer dataset by filling them with the **median income**.

Outlier Detection

- **Identifies extreme values affecting model performance**
- **Techniques:**
 - Box Plots (detects outliers)
 - ZVisualization: Scatter Plots,

- Outliers can **skew model results** and need careful handling.
- **Common detection techniques:**
 - **Box Plots:** Visualize extreme values.
 - **Scatter Plots:** Reveal unusual data points.
- **Handling outliers:**
 - Remove extreme values (if due to errors).
 - Use robust models (like Decision Trees).
- Example: Identifying outliers in **salary data** that could be errors or genuine high earners

Feature Engineering Analysis

- **Enhances model performance through feature transformation**
- **Techniques:**
 - Encoding Categorical Data (One-Hot Encoding, Label Encoding)
 - Feature Scaling (Normalization, Standardization)

- **Explanation:**
- Feature engineering enhances **model performance** by transforming raw data.
- **Techniques:**
 - **FeaEncoding Categorical Variables:**
 - One-Hot Encoding (for categorical data).
 - Label Encoding (for ordinal data).
 - **Feature Scaling:**
 - Standardization (Z-score scaling).
 - Normalization (Min-Max scaling).
- Example: Transforming **house age** into categorical bins like **Old, Medium, New**.